

עיבוד שפה טבעית - תרגיל בית 2

תיאור המשימה

בתרגיל זה תממשו Byte Pair Encoding (BPE) tokenizer כפי שנלמד בהרצאה, עבור שני דאטה סטס מדומיינים שונים. תיבחנו גם על דומיין שלישי נסתר. הטוקניזר ישמש מודל Named Entity Recognition קיים, ותעריכו את השפעתו על ביצועי המשימה.

מדד ביצועים (F1 ועילות)

מדד F1 מתייחס לדיוק F1 בינארי ברמת המילה. **יעילות הטוקניזר** מתייחסת למספר ה-tokens שכל משפט מקודד אליהם ולזמן ההסקה שלו בסביבת המכונה הוירטואלית.

מבנה הציון

סעיף	אחוז מהציון	תיאור
מימוש טוקניזרים	20%	מימוש מלא של שלושת הטוקניזרים, ועמידה ברף של ציון F1 של לפחות 0.5 על קבצי ה-dev domain_1 ו-domain_2.
תחרות	45%	תחרות משוקללת על יעילות הטוקניזר, מהירות הטוקניז וציון F1 בתיוג קבצי התחרות הנסתרים (אחד מכל דומיין).
הערכת טוקניזר 3	15%	הערכה של ביצועי הטוקניזר השלישי (הדומיין הנסתר) בדו"ח.
דו"ח הגשה	20%	כתיבת דו"ח תמציתי ועמידה מלאה בתנאי הפורמט.

קבצים מצורפים

קבצי הנתונים של התרגיל הם:

- קבצי אימון לטוקניזר: domain_1_train.txt ו-domain_2_train.txt. כל משפט מופרד בירידת שורה. אתם יכולים להשתמש בהם כרצונכם, כמו כן מצורפים קבצי dev בהתאמה.
- קבצי אימון והערכה למשימת NER (עבור מודל קיים):

○ **דומיין 1:** tagged.train_1_binary (אימון), tagged.dev_1_binary (פיתוח).

○ **דומיין 2:** tagged.train_2_binary (אימון), tagged.dev_2_binary (פיתוח).

שימו לב: אינכם נדרשים לקרוא ולעבד את קבצי ה-NER בעצמכם, אלא לוודא שבקובץ `train_ner_model.py` אתם מעבירים את הפרמטרים הנכונים בהתאם לדומיין. מלבד ניתוב הקבצים אין לשנות אף פרמטר של הרצה - הטוקנייזר שלכם יבחן אל מול מודל אחיד שהתאמן באותו אופן אצל כולם. הסבר נוסף על אופן השימוש בקבצים תוכלו למצוא בקובץ `README`.

דרישות המימוש

אימון הטוקנייזרים

- **אלגוריתם:** מימוש כל הטוקנייזרים חייב להתבסס על אלגוריתם **Byte Pair Encoding - BPE**. כלומר - הוא מתחיל מפירוק המשפט עד לרמת התו.
- **מגבלת טוקנים:** הטוקנייזר חייב לאפשר טוקנים עד רמת ה-**bigram** (שתי מילים סמוכות המקבלות token אחד). עליכם לוודא שכל טוקנייזר ייצר **לפחות bigram אחד**. אי עמידה בתנאי זה תגרור פסילת התרגיל.
- **שימוש בנתונים:** באפשרותכם להשתמש בדאטה שסופק לכם בלבד עבור האימון. **אסור** להשתמש בנתונים נוספים ממקורות חיצוניים.

ממשק ומחלקה בסיסית

- **מחלקה בסיסית:** על כל טוקנייזר לרשת מהמחלקה הבסיסית `BaseTokenizer`.
- **פונקציות:** יש לממש את כל הפונקציות הנדרשות ע"י המחלקה האבסטרקטית.
- **תו רווח:** עליכם להוסיף למחלקה את האיבר `space_token`, עם התו שבחרתם עבורו (לדוגמה, ' ' או '_').
- **שחזור:** עליכם לממש את הטוקנייזר שלכם בקובץ `code/bpe_tokenizer.py` (מחלקה היורשת מ-`BaseTokenizer`). הטוקנייזרים חייבים להיות ניתנים לשחזור באמצעות הסקריפט המסופק `generate_tokenizers.py`, המריץ את המימוש שלכם על הדאטא שניתן לכם. **אין לערוך או להגיש את הסקריפט הזה.**
- שימו לב - אתם צריכים לשמור את הטוקנייזר המאומן בפורמט הנתון, ולוודא שהסקריפטים האחרים (`test_tokenizer.py` & `train_ner_model.py`) מצליחים לקרוא אותו מבלי להכיר את המחלקה שלכם, רק את המחלקה ממנה ירשתם.

טוקנייזר ראשון ושני

השתמשו בקבצי האימון שקיבלתם (`train_1_domain.txt` ו-`train_2_domain.txt`). קבצי המשימה (NER) מגיעים מאותו הדומיין.

טוקנייזר שלישי - דומיין נסתר

השתמשו בקבצי האימון מהטוקנייזרים הקודמים. עליכם לחשוב כיצד להתמודד עם כך שהם נדגמו מדומיין שונה מזה שתבחנו עליו.

ואלידציה

עבור שני הדומיינים הנתונים, דווחו על קובץ ה-dev:

- מהירות ויעילות התיג.
- ציון ה-F1 על tagged.dev.

תחרות

שלושת הטוקנייזרים ייבחנו על קבצי תחרות נסתרים. התחרות היא משוקללת על יעילות הטוקנייזר, מהירות הטוקנייזר וציון F1. לכל רכיב משקל זהה וייתכן כי קיים טרייד-אוף בין הרכיבים האלו.

סביבת עבודה

על התרגיל לרוץ בסביבת uv שסופקה לכם (דרישות מפורטות ב-pyproject.toml). אין להתקין ספריות נוספות ללא אישור מסגל הקורס. יש לאתחל את סביבת העבודה במכונות שניתנו לכם ע"י הרצת הפקודה `bash init.sh`.

שימו לב - למכונות שניתנו לכם יש מגבלה של זמן ריצה כולל של 10 שעות, לכן עליכם לוודא שאתם משאירים אותן דולקות רק בזמן שימוש הכרחי (מומלץ לדבג את הקוד שלכם לפני שאתם מריצים על המכונה ולהריץ עליה כשאתם כבר צריכים GPU). אתם יכולים להשתמש בפקודה `time_left` בטרמינל כדי לראות כמה זמן ריצה נשאר למכונה שלכם.

הגשה

הגשת העבודה תתבצע בקובץ **zip בלבד**, בשם `HW2_123456789.zip`.

- הערכת הדו"ח תתבצע ע"ב:

- יצירתיות: בשיפור יעילות ומהירות הטוקנייזר תוך שמירה על ביצועי F1.
- כתיבה וניסוח: בהירות הדיווח בדו"ח.
- קפדנות (Rigorousity): שימוש בשיטות הערכה נכונות.
- חדשנות: רעיונות מקוריים ויישומם.

○ דרישות פורמט:

- עד עמוד אחד בגודל A4
- שוליים סטנדרטיים - 2.54 ס"מ בכל הצדדים
- פורמט של עמודה אחת
- פונט אריאל בגודל 10
- מרווח סטנדרטי בין שורות (1.15)
- מרווח של שורה ריקה בודדת בין פסקאות

- כל תמונה צריכה להיות במרכז הדף, בלי wrap text around וחייבות להיות כתוביות (caption) מתחת לתמונה
- טקסט בתוך תמונות (מקראים, כותרות וכו') חייב להיות בגודל לפחות 9 pt. אם צריך לעשות זום מעל 100% כדי לקרוא את הטקסט בתמונה, היא לא תחשב.
- שם הקובץ: `report_123456789.pdf` המכיל הסברים תמציתיים, דיווח וניתוח תוצאות.
- **תוכן חובה:** שם מחבר ות"ז, דיווח הערכת ביצועים למודל 2, הסבר על בחירת הפרמטרים, המאפיינים והשיפורים הסופיים בשני המודלים.

○ תוכן חובה:

- **אימון:** הסבר על כל טוקניזר שמומש. במה הוא שונה מהאלגוריתם הבסיסי, ואיך הוא מתמודד עם הדומיין הנסתר.
- **bigrams:** דיווח על ה-bigrams הנפוצים ביותר (עד 5) והפחות נפוצים (עד 5) בסט האימון.
- **ואלידציה:** דיווח מדד F1 על קבצי ה-dev עבור כל אחד מהמודלים, ויעילות הטוקניזרים.
- **הערכה:** הערכה שלכם עבור ביצועי הטוקניזר על הדומיין הנסתר.

- **קוד המקור שלכם** (בתיקיית code): חובה לכלול את `bpe_tokenizer.py` וכל מודול עזר שהוא מייבא, וכן סקריפטים שערכתם (למשל `train_tokenizer.py`, `test_tokenizer.py`). **אין לכלול את הקבצים** - `base_tokenizer.py`, `train_ner_model.py`, `generate_tokenizers.py`, `check_submission.py` - ההערכה מתבצעת מול הגרסאות המקוריות שלהם. הקוד צריך להיות מתועד וקריא.

- **הטוקניזרים המאומנים** (בתיקיית trained_tokenizers): שמורים בשם `tokenizer_1.pkl` (ו-2, 3) בפורמט הנתון לכם.

- שימו לב: הסקריפטים `generate_tokenizers.py` ו-`check_submission.py` מסופקים לכם - **אין לערוך או להגיש אותם**. שלושת הטוקניזרים שלכם חייבים להיות ניתנים לשחזור באמצעות `generate_tokenizers.py` על בסיס הקוד שב-code/.

שימו לב: חוסר התאמה בפורמט הקבצים פירושו ציון 0.

למען הסר ספק - מצורף לכם קובץ הגשה לדוגמה המכיל בדיוק את מבנה הקבצים שצריך להופיע ב-zip הסופי שלכם.

העתקות

חל איסור מוחלט על העברת קוד בין סטודנטים. מותר להשתמש בכלי AI (כגון chatGPT), אך הקוד הוא באחריותכם ואסור להעביר את הפרומפטים או את קטעי הקוד שנוצרו על ידי AI.